

File map & implementation order

10 first-run files + 2 deferred files. Work inside `cuboid_translation_x/`. Edit the two existing files; create the others. Reuse `physics_formula.py` and `labels/` as dependencies.

Authority: experiment rules below follow the PDF. File boundaries, API names and stacks are implementation recommendations. Unspecified numerical choices remain open.

Order / file	Responsibility
01 <code>config.py</code>	Fixed protocol; unresolved run choices; seed
02 <code>data.py</code>	CSV loading; IDs; reference groups; scaling
03 <code>neural_network.py</code> [edit]	4-input / 2-output model
04 <code>metrics.py</code>	Full-set evaluation; rulers; groups; baselines
05 <code>checkpoints.py</code>	Snapshots; absolute best; reload verification
06 <code>stopping.py</code>	Target / progress / budget decisions
07 <code>checks.py</code>	Data, scaling and learning preflight checks
08 <code>training_loop.py</code> [edit]	Batch updates; evaluations; logging; termination
09 <code>report.py</code>	Curves; failures; summary; diagnosis
10 <code>final_test.py</code>	Frozen-choice test evaluation; separate results
11 <code>animate.py</code> [deferred]	First-model demonstration after result review
12 <code>compare.py</code> [deferred]	Controlled comparisons; separate rollout study

START CHECKLIST

Open choices: stopping ruler = coarse / intermediate / fine; absolute `min_delta` in standardized validation-MSE units; record a seed. No invented defaults.

Core stack: Python standard library + NumPy + PyTorch + Matplotlib. Keep experiment settings in `config.py`; keep generated evidence in a separate run directory.

Read downward: each numbered box is one coding task. Complete its CHECK before following the arrow. Pages 2-5 continue the same sequence.

01-03 / Data contract & model

01 OPEN config.py / PROTOCOL

Stack / search: Python dataclasses; pathlib; random.seed; numpy.random.seed; torch.manual_seed.

Write: RunConfig. Inputs [v0, F, t1, t]; targets [d, v]. Train 8,000; validation 1,000; test 1,000 reserved. Fixed mass and friction. Network $4 \rightarrow 32 \rightarrow 32 \rightarrow 2$; CPU; float32.

Lock: Adam; learning rate 0.001; weight decay 0; batch 128; maximum 10,000 updates; MSE averaged over standardized outputs and samples; patience 500 updates; coverage $\geq 95\%$ on BOTH full train and validation sets. No group gate.

Rulers (d / v): coarse 0.1 m / 0.1 m/s; intermediate 0.01 m / 0.01 m/s; fine 0.001 m / 0.001 m/s. Select ONE for stopping; evaluate all three throughout.

CHECK: ruler, min_delta and seed recorded before the run. Progress clock starts at update 0; no warm-up.



02 OPEN data.py / LOAD → GROUP → SCALE

Stack / search: csv.DictReader; NumPy arrays; hashlib.sha256; torch.utils.data.Dataset / DataLoader.

Write: load_split(), attach_groups(), fit_scaler(), transform(), inverse_transform(). Map CSV columns to [v0, F, t1, t] and [d, v]; retain split + row ID. Check columns, finite values, valid times and shapes.

Reference groups: always resting: $v_0 = 0$ AND $\text{abs}(F) \leq \mu_s \times m \times g$; moved then stopped: not always resting AND reference final $v = 0$; moving at observation: reference final $v \neq 0$, including reversals.

Keep: exact reference rest/stop logic; disjoint, exhaustive groups; IDs and groups aligned through shuffling. Groups are metadata, never inputs or targets. Do not infer movement from net displacement. Existing CSVs need no overwrite.

Scale: fit on training rows only; handle near-zero scales; record the scale-handling choice. CHECK inverse recovery, [N,4] inputs / [N,2] targets, and rest/sliding/stopping/reversal/breakaway labels.



03 OPEN neural_network.py / PREDICT

Stack / search: PyTorch nn.Module; nn.Linear; nn.Sequential; nn.ReLU; nn.MSELoss.

Write: retain transmodel; linear widths $4 \rightarrow 32 \rightarrow 32 \rightarrow 2$. Recommendation: retain the existing hidden ReLU layers; the PDF does not specify activation. Inputs and outputs use the recorded normalization.

CHECK: CPU float32; batch output [N,2]; finite predictions. Inverse-scale outputs before physical errors or tolerance comparisons.

04-06 / Evaluation & stopping

04 OPEN metrics.py / MEASURE

Stack / search: torch.inference_mode; model.eval; reduction="none"; NumPy percentile; Boolean masks.

Write: evaluate(), group_metrics(), baseline_metrics(). Evaluate full train and validation with the SAME unchanged model. Standardized MSE = sum of the two squared output errors over rows / (2 × row count). Aggregate by sample counts, never average batch means.

Joint pass: $\text{abs}(d_error) < \tau_d$ AND $\text{abs}(v_error) < \tau_v$. Strict inequalities. Compute all three rates from the same predictions. Keep per-row squared errors before group aggregation.

Diagnostics: full-validation group counts, MSE, all joint pass rates; physical d/v MAE, P95 and maximum errors. Report overall physical errors too. Empty group: count 0, metrics undefined.

Extra slice: $v_0 = 0$ AND $\text{abs}(\text{abs}(F) - 3.924 N) \leq 0.2 N$; overlaps groups; no acceptance gate. At update 0 score [0,0] and $[v_0 \times t, v_0]$ baselines. Keep per-row errors at updates 0 and 10, if reached.



05 OPEN checkpoints.py / PRESERVE

Stack / search: torch.save / torch.load; state_dict; Python json; RNG getstate / setstate.

Write: save_checkpoint(), load_checkpoint(), save_if_best(). Save every scheduled evaluation, final and selected models. Store weights, model config and normalization; optimizer and RNG states for continuation. Recommend recording update, epoch and sampler position for continuation.

Absolute best: save every new validation-MSE minimum, even an improvement smaller than min_delta. Keep this best separate from the progress reference. CHECK reload reproduces predictions.



06 OPEN stopping.py / DECIDE

Stack / search: Python dataclass; Enum; state machine. Write StopState and check_stop(); return decisions to the checkpoint writer.

State: progress-reference MSE and update; separate absolute-best MSE/checkpoint. Initialize progress reference at update 0. Inputs: update, matched overall train/validation selected-ruler rates, validation MSE.

Exact check order: finite metrics → save new absolute best → target → update progress reference → patience → cap. Numerical failure aborts separately; never success.

Target: both rates ≥95%, including update 0: TARGET_REACHED; save/select the first-passing checkpoint. Otherwise meaningful progress iff $\text{current_val_mse} < \text{reference_mse} - \text{min_delta}$; replace reference and reference update only then.

Stop: elapsed updates since reference ≥500: NO_PROGRESS. At 10,000: MAX_UPDATES. If both, record both flags; primary NO_PROGRESS. Select lowest-validation-MSE checkpoint unless target reached. Small gains accumulate against the unchanged reference; stops occur only at checks.

07-09 / Check, train & diagnose

07 OPEN checks.py / PREFLIGHT

Stack / search: Python unittest; numpy.testing.assert_allclose; torch.isfinite; tensor.clone.

Write: check_data(), check_scaling(), check_learning(). Check data and reference cases; training-only scaler and inverse recovery; one update changes weights with finite loss/gradients; substantially reduce error on 32-64 fixed examples.

CHECK: small fit is an implementation check, not generalization evidence or a perfect-boundary-fit requirement. Start fresh afterward. Recommended stop tests: target at 0; strict tolerance boundary; sub-min_delta accumulation; progress at 200 then stop at 700; patience + cap; numerical abort.



08 OPEN training_loop.py / ORCHESTRATE

Stack / search: torch.optim.Adam; DataLoader(shuffle=True, drop_last=False); time.perf_counter; csv.DictWriter.

Setup: seed → load train/validation → fit scaler → fresh model + Adam → evaluate update 0 → checkpoint + stop check. Keep test reserved.

Update: batch → predict → MSE → clear gradients → backward → optimizer update → increment global step. Shuffle each epoch; keep partial batch and already-passing samples. Record batch loss each update; abort numerical failures separately.

Evaluate: updates 0, 10, 20, 50, 100; then every 100; also normal termination. Use full matched train/validation metrics, never the latest batch pass rate. Invoke the ordered checkpoint/stop pipeline; restore train mode if continuing.

CSV: update, epoch, sample exposures, elapsed time, split, MSE and three pass rates; group metrics/counts; physical errors; breakaway slice. Retain raw observations. Count actual rows seen, including partial batches.

CHECK: first possible stagnation stop = update 500; evaluate and stop at 10,000; never run update 10,001. At exit preserve final, selected and last valid evidence; record reason and all stop flags.



09 OPEN report.py / CURVES → FAILURES → REVIEW

Stack / search: Matplotlib pyplot.subplots / Axes.plot; csv; NumPy percentile; json. Write plot_curves(), failure_table(), summarize_run().

Plots: validation MSE and pass rates versus updates; matched train/validation curves; full-validation group curves with counts. Pass-rate axis 0-100%. Preserve raw trends; require neither a logarithmic shape nor a plateau.

Failure tables: selected AND last valid checkpoint; row ID, input, reference group, target, prediction, physical errors, severity = max(abs(d_error)/tau_d, abs(v_error)/tau_v). Use selected ruler; keep physical units. Failed-only counts/errors; representative and worst cases. Zero failures: count 0, failed-only statistics undefined.

Run record: config, data hashes/row IDs, seed, software/device, normalization, updates/exposures/time, termination. Summary: ruler/coverage, measured rates, groups, physical errors, baselines, reason and limits.

Review: raw trend + stop settings → matched train/validation → failing groups/examples → one explanation + one controlled follow-up. Use validation; no automatic model enlargement or longer training. Timing is supporting context.

10-12 / Freeze, demonstrate & compare

10 OPEN `final_test.py` / FROZEN CHOICE ONLY

Stack / search: reuse `data.py`, `metrics.py` and `checkpoints.py`; Python `json`; `torch.inference_mode`.

Write: `evaluate_frozen_choice()`. Freeze the training protocol and model selection BEFORE opening the reserved 1,000 test rows for evaluation. Reuse training-fitted normalization and the selected checkpoint.

Report: separate test metrics; no tuning against test results. Training target attainment is distinct from final test acceptance; the PDF supplies no separate numerical test-acceptance threshold.

CHECK: label the scope as synthetic one-dimensional reference agreement, not general 3D or real-world validation. If validation follow-ups are still changing the choice, defer this test execution.



11 OPEN `animate.py` / SECOND MILESTONE

Stack / search recommendation: Matplotlib animation.`FuncAnimation`; saved PyTorch `state_dict`; analytical reference.

Gate: review the first training result before animation design. Build the first-model demonstration before other experiments; Models B/C are not prerequisites.

Write later: load saved training checkpoints + recorded normalization; obtain corresponding analytical-reference predictions; implement the reviewed demonstration design.

CHECK: use the first model and its saved training progress. Animation design remains deferred as required by the PDF.



12 OPEN `compare.py` / AFTER THE DEMONSTRATION

Stack / search: Python `dataclasses.replace`; NumPy subset indexing; reuse training/evaluation/report modules; Matplotlib.

Write later: `run_architecture_comparison()`, `run_data_comparison()`, `run_loss_comparison()`, `study_rollouts()`. Change one main factor at a time; preserve matched data and protocols.

Architectures: $4 \rightarrow 16 \rightarrow 2$; $4 \rightarrow 32 \rightarrow 2$; $4 \rightarrow 16 \rightarrow 16 \rightarrow 2$; $4 \rightarrow 32 \rightarrow 32 \rightarrow 2$. Repeat promising comparisons with three seeds.

Data: nested 500 / 1,000 / 2,000 / 4,000 / 8,000 training rows; fresh models; subset-specific normalization. Loss comparisons are deferred; candidate losses are not specified by this PDF.

Rollouts: separate study; matched force schedules; explicit position tracking; out-of-range checks. The PDF refers to supporting notes for comparison limitations; do not invent additional comparison standards in this manual.

CHECK: first-run training + evidence $\rightarrow$ result review $\rightarrow$ first-model animation $\rightarrow$ controlled comparisons. Keep final test evaluation tied to a frozen choice, not a recurring tuning loop.